CHAPTER 4

HEURISTICS

4.1 INTRODUCTION

Decisions pertaining to the strategic and tactical level aspects might not be dynamic in nature. Nevertheless, decisions have to be taken within a reasonable amount of time. Owing to the NP-complete nature of the problem, we saw in the previous chapter that exact methodologies fail to arrive at an optimum solution within computational times deemed practical. In this chapter, we propose two heuristics to solve the integrated LAP. The first heuristic is designed based on Dijkstra's algorithm whereas the second heuristic uses ant colony optimization (ACO)-based approach to arrive at a solution.

4.2 DIJKSTRA'S ALGORITHM BASED HEURISTIC

4.2.1 Input Parameters

We design the heuristic as a series of least cost path problems for each chosen locomotive that needs to be solved in such a way that each locomotive serves as many trains as possible and at the same time the total number of locomotives is also minimized. To solve the problems as lowest cost path problem, the arcs in the network need to have costs associated with them. Since not all arcs have costs or distance associated with them, there is a need for proxy costs. Hence, we assign proxy costs to all arcs.

The priority is to ensure that each locomotive serves as many train arcs as possible. Accordingly, TrArcs are assigned the least cost. These arcs are assigned costs equal to the ranks of actual costs when they are arranged in ascending order of their actual costs. Once serviced by a locomotive, the costs of TrArcs are increased marginally but higher than the cost of other un-serviced TrArcs. This is to ensure that the possibility of selection of such TrArcs for deadhauling of locomotive remains open. Once two locomotives serve any TrArc, the cost is increased to infinity to avoid the selection of such TrArcs.

Each TrArc is preceeded and succeeded by CDArcs. They are assigned the next lowest cost to ensure that whenever a TrArcs is chosen, other incoming/ outgoing Arcs are avoided.

Waiting Arcs at the shed and at stations are assigned costs which are proportional to the duration of the time (difference between arrival time and departure time of the arc).

Deadheading arcs originating either at the shed or stations but terminating only at a station are assigned the next lowest cost. Deadheading arcs terminating at the sheds are given very high costs so as to avoid locomotive trips to sheds unless absolutely necessary. Such cases include situations when the locomotive is due for periodic maintenance, the locomotive trip has reached the end of the planning horizon or when a direct link between two stations is absent and the locomotive has to take a detour through a locomotive shed to reach some other station where a train is waiting to be serviced.

Trip maintenance and periodic maintenance arcs are given the highest costs. Since the algorithm needs to avoid these arcs unless the locomotives are due for maintenance.

4.2.2 Locomotive Selection

Depending on its type, hp rating and top speed, the procurement cost of each locomotive varies. Capital investment represents the major cost in our objective function. Further, depending on the route and distance travelled, the operating cost associated with each locomotive varies while servicing any particular TrArc or DArc or D`Arc. This cost also depends on the age of the locomotive as older locomotives tend to consume more power/fuel depending on whether it is electric/ diesel and hence are prone to be less efficient. We also assign penalty costs to locomotives servicing certain TrArcs when a particular type of locomotive is preferred less/ not preferred to service the corresponding TrArc. Given this set of costs, we need to select one locomotive at a time which will service as many TrArcs as possible, while adhering to the constraints and return back to its home shed at the end of the planning horizon. We calculate the total cost of each locomotive by adding the respective annuitized capital investment cost to the operating costs and penalty cost of all the arcs. We then use a greedy approach of selecting the locomotive with the lowest total cost first. Once the locomotive has served a set of TrArcs as allotted by the algorithm, we replace the cost with infinity and then choose the second lowest cost locomotive.

4.2.3 Dijkstra's Function

The Dijkstra's Algorithm Function used in this algorithm is an adaptation of the traditional Dijkstra's algorithm for the shortest path problem with the function modified to suit the requirements specific to our problem. The traditional Dijkstra's algorithm starts at the start node and visits all the nodes on its way to find the shortest path/ lowest cost path to the end node. Once all the nodes have been visited, the Dijkstra's algorithm terminates. This approach however, is not suited to our problem. In the Dijkstra's algorithm function, we attempt to find the least cost path for each locomotive from the start node, which is the home shed at the beginning of the planning horizon, to the end node, which again is the home shed, albeit at the end of the planning horizon. Since the network is a space-time network, once we move ahead in time, we cannot travel back in time visit an unvisited node at an earlier point of time. Consequently, we apply a different termination rule for this function. We terminate the function when one of the following conditions is met:

1. Locomotive is due for periodic maintenance
2. Locomotive is due for trip maintenance
3. Locomotive is due for both, trip and periodic maintenance
4. Locomotive has reached the end of the planning horizon

The path updation rule too is different for the function as compared to the original Dijkstra's algorithm. The path from the start node to each of the other nodes is updated only if the new path has a lower cost/distance as compared the old path. In contrast, since the aim is for each locomotive to serve as many trains as possible and since distance is associated only with TrArcs, we update the path if the distance

of the new path is greater than the old path irrespective of the cost to reach that node. If the distance of the new path is equal to the old path, then the path is updated if the cost of the new path is less than the cost of the old path. The algorithm for Dijkstra's function is given in the flowchart below:

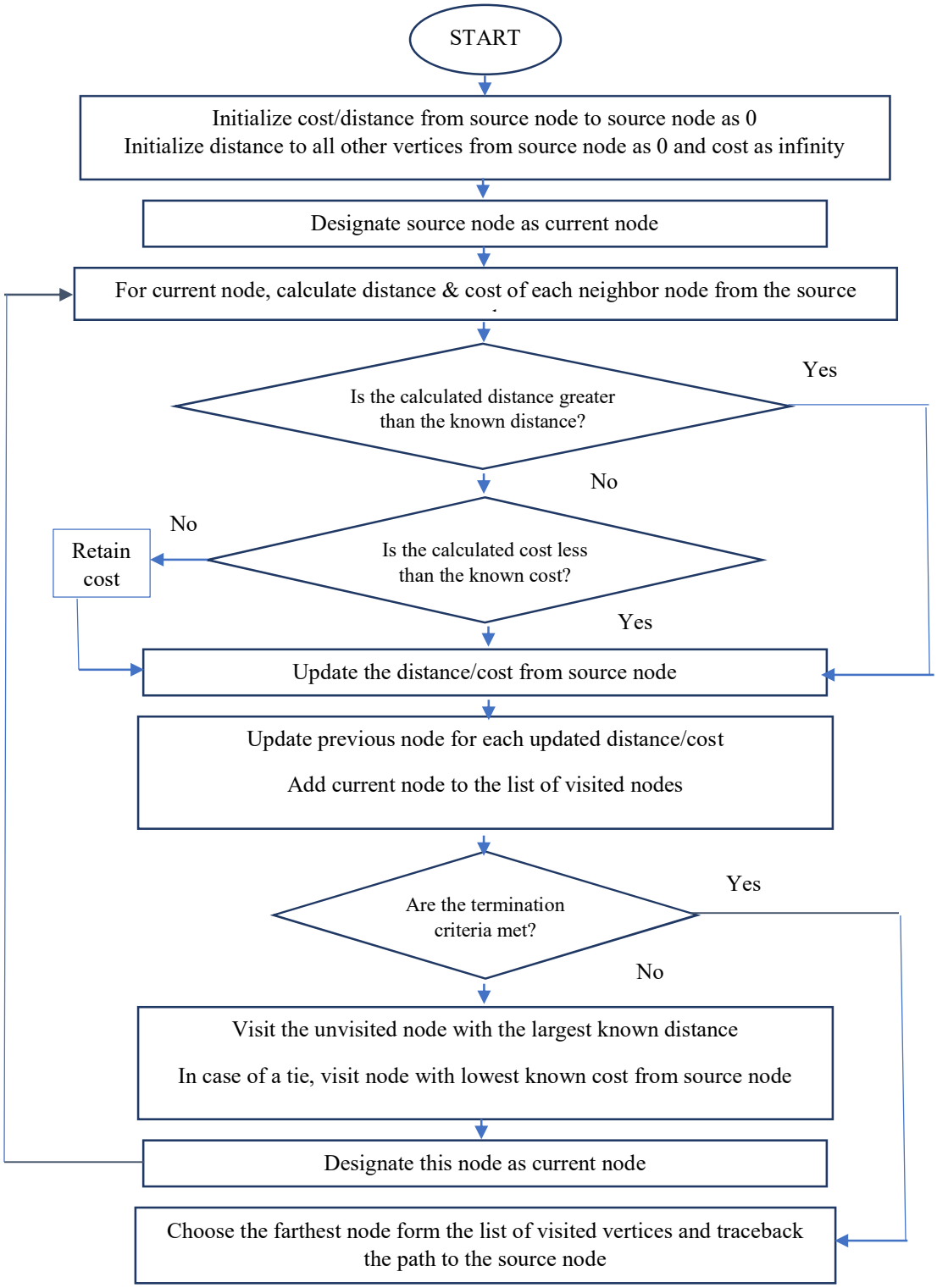

Figure 7: Algorithm for Dijkstra's Function

4.2.4 Algorithm

The algorithm for the heuristic is given in the flowchart below. Each locomotive is selected based on the rules as explained in section 4.3.2. Post identification of the lowest total cost locomotive, we use Dijkstra's function explained in section 4.3.3 to identify the lowest cost path for the locomotive. Based on the position of the locomotive in the space-time network, restart node is calculated and the procedure repeated till the locomotive reaches the end of the planning horizon. Restart node for a locomotive depends on the type of maintenance undergone by the locomotive. If the function is terminated as a result of locomotive being due for trip maintenance, restart node is the same station at which the locomotive ended its trip previously but after a time duration of two hours. Similarly, if the function is terminated as a result of locomotive being due for periodic maintenance, restart node is the home shed of the locomotive after such time as is needed for the locomotive to head back to the home shed and undergo the preventive maintenance. At the end of the planning horizon, the algorithm checks for un-serviced TrArcs. Existence of un-serviced TrArcs necessitates selection of a new locomotive and a repetition of the above procedure. The heuristic is terminated when all TrArcs have been serviced by locomotives.

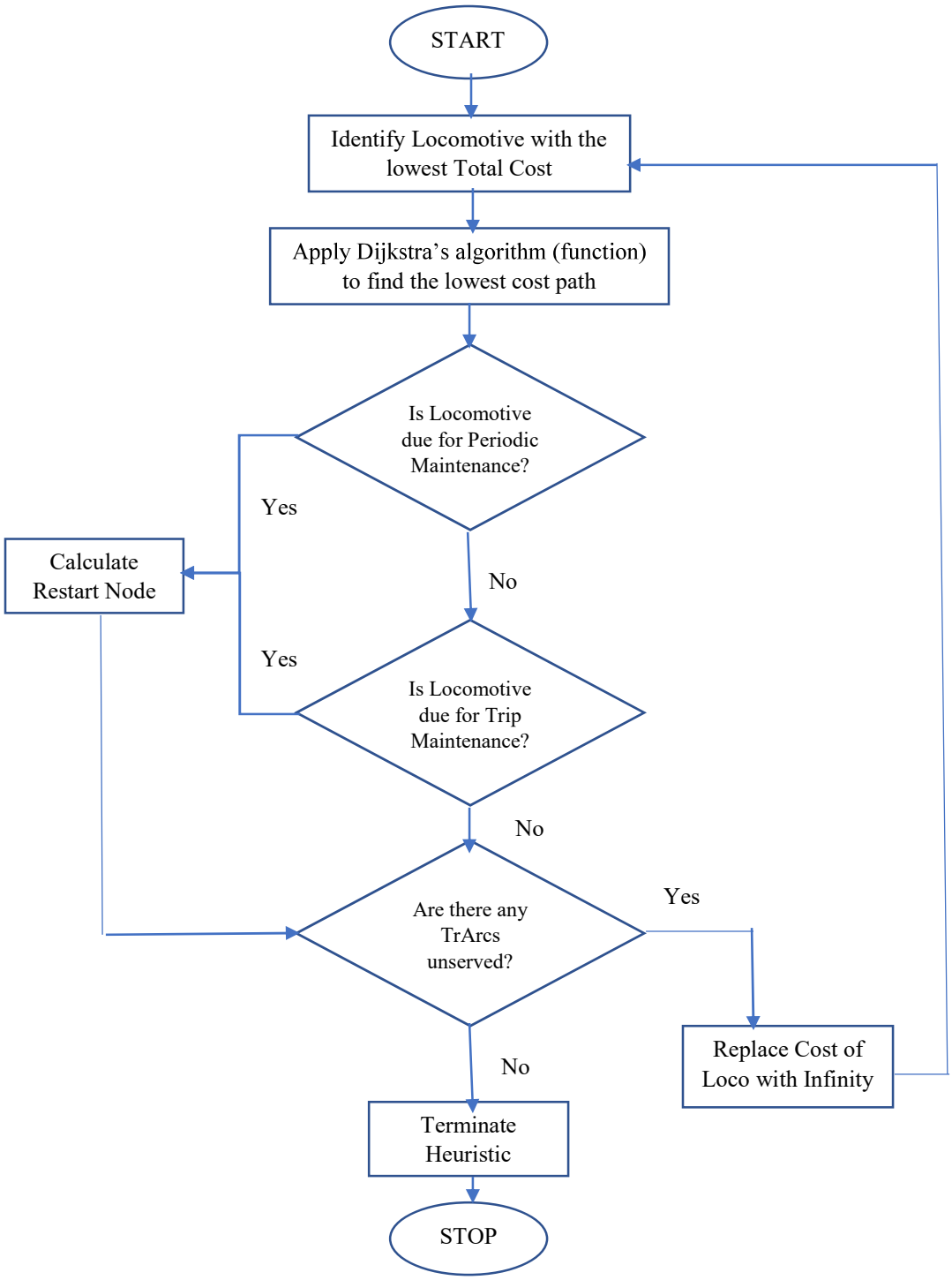

Figure 8: Algorithm for Dijkstra's Algorithm-based Heuristic

4.3 ANT COLONY OPTIMIZATION BASED HEURISTIC

Ant Colony optimization is a technique for optimization that was introduced in early 1990's. ACO algorithm models the behaviour of real ant colonies in establishing the shortest path between food sources and nests and this technique is applied on number of combinatorial optimization problem. In this section, we design an ACO based algorithm to solve the LAP.

4.3.1 Input Parameters

TrArcs, DArcs and DlArcs are the only that have costs and distance associated with them. As a result, there is a need for proxy costs. We assign proxy costs to all arcs. The priority is to ensure that each locomotive serves as many train arcs as possible. To ensure this, we need to ensure that TrArcs have the maximum amount of pheromones and consequently TrArcs are assigned the least cost. These arcs assigned costs equal to the ranks of actual costs when they are arranged in ascending order of their actual costs. Once serviced by a locomotive, the costs of TrArcs are increased to a level higher than the cost of other un-serviced TrArcs. This ensures that the un-serviced TrArcs will still have a higher probability of selection on account of higher pheromone levels, but the possibility of selection of serviced TrArcs for deadhauling of locomotive remains open as well. Once, 2 locomotives serve any TrArc, the cost is increasing the cost to infinity is not sufficient to avoid the selection of such TrArcs.

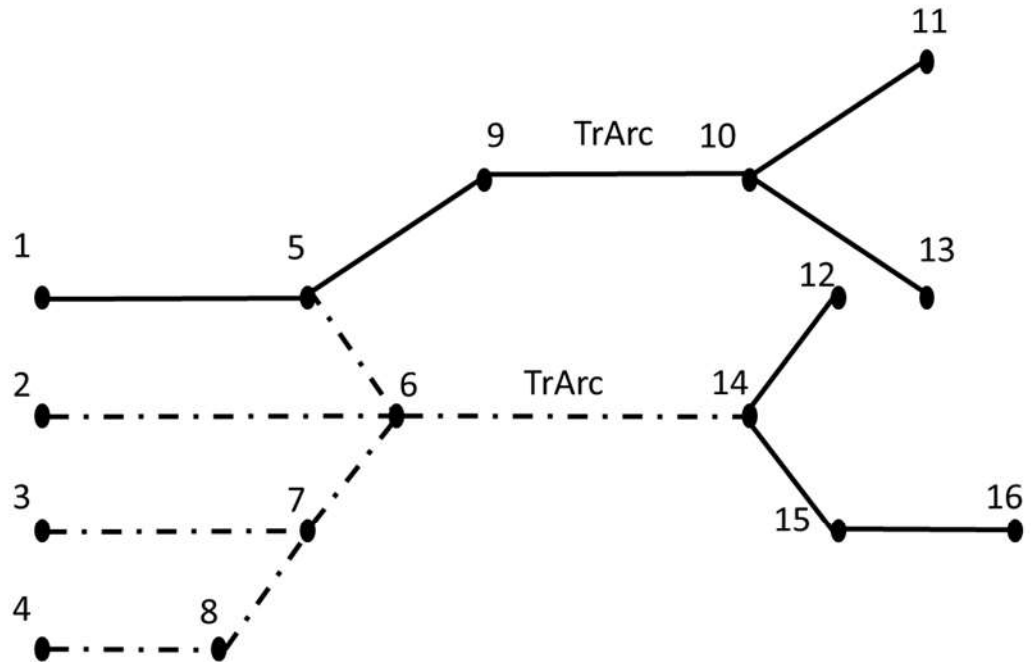

Figure9: Space-Time Network Representation for ACO

Consider a portion of the space-time network as shown in the figure. Arcs 9-10 and 6-14 are TrArcs. Assume that TrArc 6-14 has already been served by 2 locomotives. Unlike the Dijkstra's algorithm function, in the ACO function, assigning a cost of infinity to TrArc 6-14 would be insufficient from preventing the future locomotives to serve the TrArc. For eg. any locomotive arriving at node 6 has only one route to go forward. Hence, Arc 6-14 has a relative probability 1 of being chosen as the outgoing arc from node 6. Consequently, we need to ensure that all paths leading up to node 6 have zero pheromone levels so that arc 6-14 is not chosen. Accordingly, all arcs on the paths 4-8-7-6-14, 3-7-6-14, 2-6-14 are reassigned zero pheromone levels. On path 1-5-6-14, arc 1-5 is not assigned zero pheromones since at node 5; there is another outgoing arc 5-9 which does not lead to TrArc 6-14. Hence, on path 1-5-6-14, only arcs 5-6 and 6-14 are assigned zero pheromone levels. In the network diagram, the arcs with zero pheromone levels are represented

by hashed lines whereas the solid lines represent arcs with some amount of pheromones.

Prior to servicing a train and post reaching the destination, each locomotive has to couple and decouple itself respectively from the train. So, in the time-space network, each TrArc is preceeded and succeeded by CDArcs. To ensure that whenever a TrArc is chosen, CDArcs are preferred over other incoming/ outgoing Arcs, they are assigned the next lowest cost so as to ascertain that CDArcs have the next highest level of pheromones.

For Waiting Arcs at the shed and at stations costs (pheromones) are assigned proportional (inversely proportional) to the duration of the time (difference between arrival time and departure time of the arc).

Deadheading arcs originating either at the shed or stations but terminating only at a station are assigned the next lowest cost. Deadheading arcs terminating at the sheds are given very high costs or negligible levels of pheromones so as to avoid locomotive trips to sheds unless absolutely necessary. Such cases include situations when the locomotive is due for periodic maintenance, the locomotive trip has reached the end of the planning horizon or when a direct link between two stations is absent and the locomotive has to take a detour through a locomotive shed to reach some other station where a train is waiting to be serviced.

Trip maintenance and periodic maintenance arcs are given the highest costs or the least levels of pheromones since the algorithm needs to avoid these arcs unless the locomotives are due for maintenance.

4.3.2 Locomotive Selection

The locomotive selection procedure follows the same logic outlined in the section 4.2.2 earlier.

4.3.3 ACO Funciton

In the ACO function, an individual ant simulates a locomotive. The chosen locomotive's route is constructed by incrementally selecting stations until the status of the locomotive necessitates maintenance or the locomotive has reached the end of the planning horizon. Each ant starts at the source node defined by the heuristic, which, at the beginning of the planning horizon is the shed node. At intermediate stages, the source node could be either the stations or the sheds, depending on the node at which the ACO function terminated at the previous stage. The next node to be visited by the ant is selected from one among the neighbours of the source node. The total distance is computed based on the number TrArcs served. Time elapsed is computed based on the position of the ant in the time-space network. This continues till one of the termination criteria is met.

Using ACO, each ant constructs a locomotive route that serves as many TrArcs as possible. To select the next node j, the ant uses the following probabilistic formula

$$p_{ij} = \frac{(\tau_{ij})(\eta_{ij})^{\beta}}{\sum_{u}(\tau_{iu})(\eta_{iu})^{\beta}}$$

where τ_{iu} is equal to the amount of pheromone on the path between the current node i and neighbouring nodes u. The value η_{iu} is defined as the inverse of the distance between the two nodes and the parameter β establishes the importance of distance in comparison to pheromone quantity in the selection algorithm. Arcs already served by the ant are stored in an array. The ant will return to maintenance shed or home shed depending on whether the maintenance conditions are met or the planning horizon has ended.

Trail updating is conducted by reducing the amount of pheromone on all visited arcs in order to simulate the natural evaporation of pheromone and to ensure that no one path becomes too dominant. This is done with the following trail updating equation,

$$\tau_{ij} = (1-\alpha)\tau_{ij} + (\alpha)\tau_0$$

where α is a parameter that controls the speed of evaporation and τ_0 is equal to an initial pheromone value assigned to all arcs in graph G. DArcs, DlArcs and TrArcs which have already met the bounds on equipment numbers are updated with zero pheromone levels. The algorithm for the ACO function is given in the flowchart below:

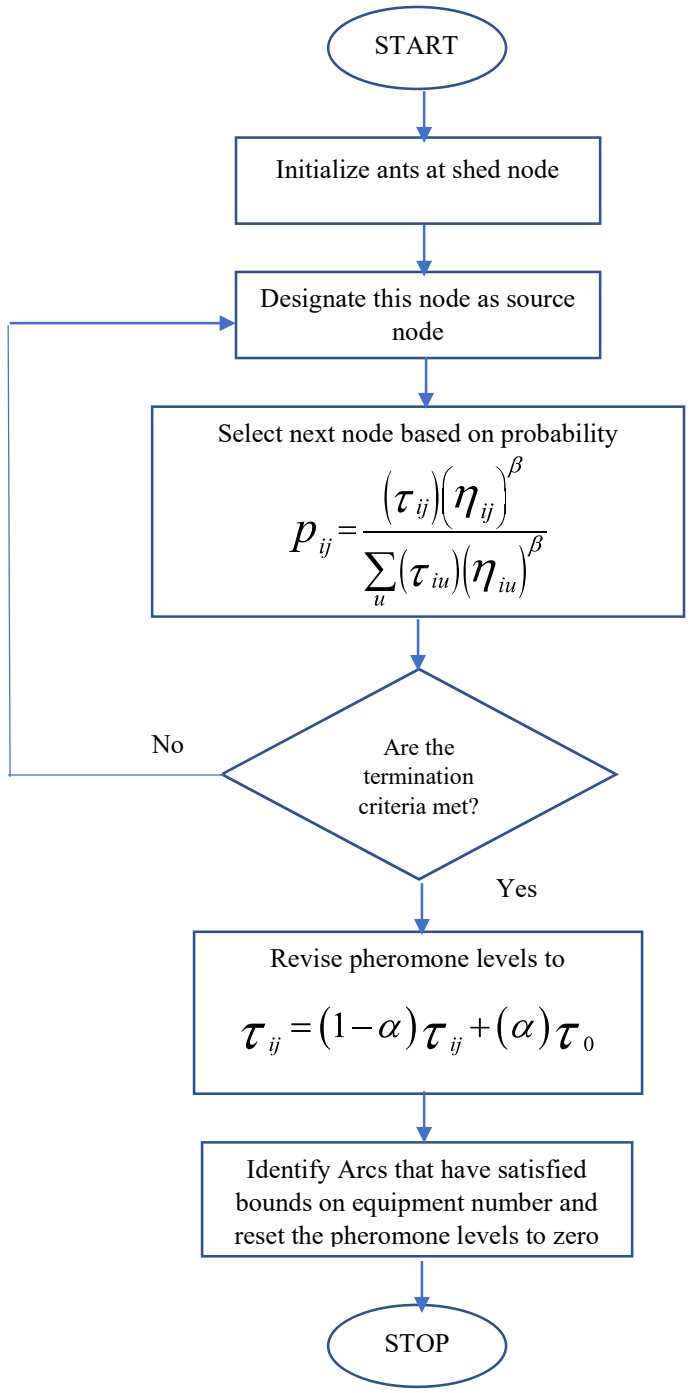

Figure 10: Algorithm for the ACO function

4.3.4 Algorithm

The algorithm for the heuristic is given in the flowchart below. Each locomotive that is simulated by the ant is selected based on the rules as explained in section 4.2.2. Post identification of the lowest total cost locomotive, we use ACO function explained in section 4.2.3 to identify the lowest cost path for the locomotive. Based on the position of the locomotive in the time-space network, restart node is calculated and the procedure repeated till the locomotive reaches the end of the planning horizon. At the end of the planning horizon, the algorithm checks for un-serviced TrArcs. Existence of un-serviced TrArcs necessitates selection of a new locomotive and a repetition of the above procedure. The heuristic is terminated when all TrArcs have been serviced by locomotives.

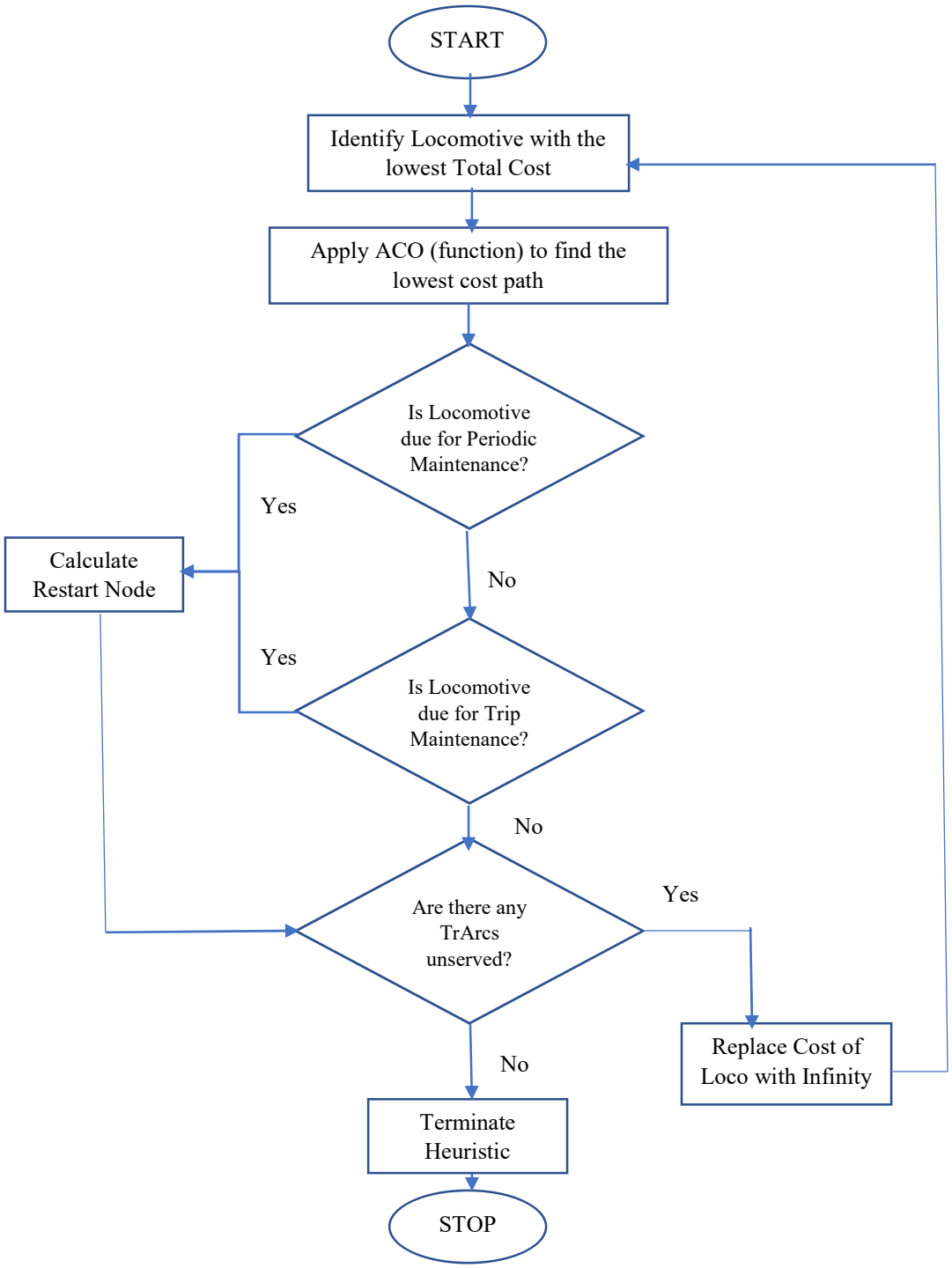

Figure 11: Algorithm for the ACO-based Heuristic

4.4 RESULTS AND DISCUSSIONS

We use the same set of 25 problem instances that were used to test the mathematical models. For the Dijkstra's algorithm-based heuristic, we were able to arrive at solutions within a reasonable amount of time. The quality of the solution also is reasonably good for the given test problem instances. In contrast, although, we were able to arrive at solutions within a reasonable amount of time, the quality of the solution is is not very encouraging even for the test problem instances. A summary of the test results is given in table 10 below.

The difference in the quality of the solutions of Dijkstra's algorithm-based heuristic and ACO-based heuristic could be attributed to the fact that Dijkstra's algorithm is a polynomial time algorithm for the least cost path problem. Whereas, ACO is a heuristic approach to find the least cost path for a problem. Since, the locomotive selection procedure is same for both our heuristics, the difference boils down to how we find the shortest path for each locomotive.

Sr. No.	No. of loco types	No. of stations	Planning horizon	No. of Trains	Time gap between two consecutive time slots	Dijkstra's Termination time (sec)	Optimality gap	ACO Termination time (sec)	Optimality gap
1	2	2	1 day	2	0.5	2	0.00%	8	3.73%
2	2	2	3 days	12	0.5	4	0.00%	31	7.35%
3	2	2	4 days	16	0.5	2	0.00%	4	3.48%
4	2	3	2 days	12	0.5	6	0.00%	48	12.19%
5	2	4	2 days	14	0.5	5	0.00%	15	3.94%
6	2	3	3 days	18	0.5	16	0.00%	1639	1.96%
7	2	2	22 days	24	1	13	0.00%	280	11.48%
8	2	4	20 days	28	1	69	10.78%	365	36.87%
9	2	3	14 days	36	1	48	0.00%	2307	31.92%
10	2	4	5 days	28	0.5	35	0.18%	511	0.00%
11	2	3	4 days	36	0.5	28	18.13%	2922	29.31%
12	2	12	48 days	36	4	346	0.92%	317	0.00%
13	2	12	48 days	42	4	428	0.88%	885	42.59%
14	5	12	48 days	42	4	468	4.83%	693	25.25%
15	2	12	48 days	48	4	838	0.00%	1777	11.47%
16	2	14	48 days	42	4	837	13.06%	556	0.00%
17	5	14	48 days	42	4	876	0.00%	700	36.94%
18	5	12	48 days	36	4	508	0.00%	342	18.93%
19	2	6	48 days	36	4	156	0.00%	463	9.96%
20	5	6	48 days	36	4	146	0.00%	487	5.71%
21	2	6	48 days	36	4	145	0.00%	485	5.13%
22	5	10	48 days	36	4	276	0.22%	335	0.47%
23	2	10	48 days	36	4	277	7.80%	377	19.43%
24	2	8	48 days	36	4	185	7.30%	518	43.62%
25	5	8	48 days	36	4	178	4.50%	514	48.85%

Table 10: Summary of Computational Tests – Heuristics